

VITyarthi — Build Your Own Project

PROJECT REPORT

SpendSense

Smart Student Expense Tracker & Spending Analytics

*A natural-language expense logger with behavioural analytics,
statistical anomaly detection, and budget control for college students*

Submitted by: Aryan Patil

Registration No.: 25BAI10768

Course: Programming in Java

University: VIT Bhopal University

Date of Submission: 18 September 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
 - 6.1 Use Case Diagram
 - 6.2 Workflow / Process Flow Diagram
 - 6.3 Sequence Diagram
 - 6.4 Class / Component Diagram
 - 6.5 ER Diagram (Storage Design)
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

SpendSense is a client-side web application that helps college students understand and control their day-to-day spending. It was developed for the VITYarhi flipped-course evaluation as an original project applying the software design, requirements engineering, data-processing, and analysis concepts covered in the syllabus.

The defining idea behind the project is that the hardest part of personal expense tracking is not analysis — it is data entry. A student who buys a ₹40 bus ticket will not open an application and complete a five-field form. SpendSense therefore reduces logging to a single sentence typed the way a student would actually say it, for example "₹120 lunch at college canteen", and infers the amount, the category, and the record structure automatically through a regex tokenisation and keyword-mapping engine.

Once data is captured with low friction, the application converts that history into interpretable insight rather than raw tables. It renders a colour-graded heatmap calendar of daily spending velocity, assigns the student a data-driven Spending Personality with a transparent explanation, flags statistically unusual transactions through a rule-based anomaly detector, and maintains per-category budgets with a recommended daily allowance for the remaining days of the month.

This report documents the problem addressed, the functional and non-functional requirements, the system architecture and design artefacts, the implementation approach, the testing carried out, and the challenges, learnings, and future direction of the project.

2. Problem Statement

College students manage small, frequent, cash- and UPI-heavy expenses — canteen meals, bus fares, photocopies, hostel bills — that individually feel trivial but collectively consume the bulk of a monthly allowance. Most students consequently reach the end of the month unable to explain where their money went.

Existing expense-tracking applications fail this user group for three distinct reasons:

- **Logging friction.** Conventional trackers require a multi-field form for every transaction. Because the effort of recording a ₹40 expense is disproportionate to its value, logging is abandoned within days and the dataset never becomes complete enough to be analytically useful.
- **Generic categories.** Mainstream applications are categorised around adult spending — mortgages, insurance, fuel, groceries. They have no representation for academic supplies, hostel bills, or campus food, so a student's spending collapses into an uninformative "Other" bucket.
- **Data without insight.** Even where data is captured, most tools stop at a pie chart. They report what was spent but never tell the student what their pattern reveals about their habits, which specific transactions were genuinely abnormal, or how much they can safely spend tomorrow.

SpendSense addresses all three failures simultaneously: it removes entry friction through natural-language logging, classifies spending into eight categories designed specifically around student life, and converts the resulting history into behavioural insight through a personality engine, a statistical anomaly detector, a daily-velocity heatmap, and a forward-looking budget allowance calculator.

3. Functional Requirements

The system is built around three major functional modules, supported by additional analytics and reporting capabilities. Each module has a defined input/output contract, and together they form a closed workflow: capture feeds analysis, and analysis feeds budgeting guidance that influences the next capture.

3.1 Module 1 — Expense Capture & Classification

- **FR-1.1:** The system shall accept freeform natural-language expense entries and extract the monetary amount using regex tokenisation. Examples handled include "₹120 lunch at college canteen", "40 bus ticket", "250 xerox and textbooks", and "1800 hostel wifi bills".
- **FR-1.2:** The system shall classify each entry into one of eight student-tailored categories using a keyword-mapping engine: Food & Dining, Transportation, Academic Supplies, Entertainment, Shopping, Health, Bills & Utilities, and Other.
- **FR-1.3:** The system shall provide a structured manual-logging tab as an alternative entry path, capturing amount, date, category, payment method, and an optional note.

- FR-1.4: The system shall validate all inputs — rejecting non-positive amounts, malformed dates, and empty required fields — and display a clear error without discarding the user's partial input.
- FR-1.5: The system shall record the payment method for each transaction from the set UPI, Cash, Debit Card, Credit Card, and Net Banking.

3.2 Module 2 — Temporal Visualisation & Behavioural Analytics

- FR-2.1: The system shall render a monthly heatmap calendar colouring each day by total spend across a five-tier intensity scale: None; Low (up to ₹150); Moderate (up to ₹400); High (up to ₹850); and Surge (above ₹850).
- FR-2.2: The system shall provide an interactive day inspector allowing the student to click any calendar cell and view that day's full transaction breakdown.
- FR-2.3: The system shall compute and display derived behavioural statistics: peak outflow day, most frugal day, active consecutive logging streak, and the weekend-versus-weekday spending split.
- FR-2.4: The system shall assign a Spending Personality profile derived from category expenditure ratios, selecting from The Campus Foodie, The Academic Striver, The Daily Commuter, The Entertainment Enthusiast, The Disciplined Saver, and The Balanced Manager.
- FR-2.5: The system shall display a transparent rationale explaining which category ratios triggered the assigned profile, accompanied by actionable advice.

3.3 Module 3 — Budget Control & Anomaly Detection

- FR-3.1: The system shall compute a per-category baseline mean and standard deviation from recorded history and flag transactions that deviate beyond the configured threshold.
- FR-3.2: Each anomaly alert shall state the exact multiplier over the category baseline and the surplus amount, phrased analytically rather than judgementally — for example, a ₹2,450 electronics purchase measured against a ₹180 academic-supplies baseline.
- FR-3.3: The system shall maintain per-category monthly budget limits and display actual expenditure, remaining balance, and utilisation percentage for each.
- FR-3.4: The system shall render a colour-coded status badge per category with four states: Safe, Moderate, Near Limit, and Exceeded.
- FR-3.5: The system shall calculate a recommended daily spend allowance from the remaining balance divided by the days remaining in the current month.
- FR-3.6: The system shall support inline editing of monthly category limits and a one-click reset to college-student default values.

3.4 Supporting Capabilities — Analytics & Reporting

- FR-4.1: The system shall render an interactive SVG donut chart of category distribution with slice highlighting on hover or selection.
- FR-4.2: The system shall render a daily spending velocity bar chart and a payment-method distribution breakdown.
- FR-4.3: The system shall generate a formal financial report suitable for printing as a statement.
- FR-4.4: The system shall export the full transaction history to CSV and import transactions from a previously exported CSV file.
- FR-4.5: The system shall persist all data locally so that records survive page refreshes and browser restarts.

3.5 Input / Output Structure

Module	Input	Output
Expense Capture	Freeform text or structured form fields	Validated Expense record with inferred category
Heatmap Calendar	Expense history for the selected month	Colour-tiered calendar grid, day-level breakdown, streak and split statistics
Personality Engine	Category-wise expenditure ratios	Assigned profile, triggering rationale, advice
Anomaly Detector	Category history (mean, std-dev) and a new expense	Alert list with multiplier and surplus values

Module	Input	Output
Budget Manager	Category limits and month-to-date spend	Utilisation %, status badge, recommended daily allowance
Reports	Full transaction history	SVG charts, printable statement, CSV file

4. Non-Functional Requirements

The following non-functional requirements were specified and designed for, exceeding the minimum of four required by the project brief.

#	Requirement	Specification
NFR-1	Performance	Natural-language parsing and category inference complete within 50 ms per entry; dashboard recomputation across a full year of transactions completes within 300 ms, keeping the interface responsive during typing.
NFR-2	Security & Privacy	All financial data remains on the student's own device in LocalStorage. No transactions are transmitted to any server, no account or credentials are required, and no third-party analytics are embedded — eliminating the disclosure risk inherent in cloud-hosted finance apps.
NFR-3	Usability	Any expense can be logged in a single sentence with one keystroke to submit. All primary views — logger, calendar, budget, analytics — are reachable within two clicks from the dashboard. Categories carry both colour and emoji so status is never conveyed by colour alone.
NFR-4	Reliability	Parser failure is non-destructive: if an amount cannot be extracted, the entry is redirected to the manual form with the text preserved rather than silently dropped. Corrupt or partially written LocalStorage records are skipped on load rather than crashing the application.
NFR-5	Maintainability	Strict separation between components (presentation), services (logic engines), models (types), and utils (helpers). Category definitions, intensity thresholds, personality rules, and anomaly thresholds are declared as configuration constants, so tuning behaviour requires no changes to rendering code.
NFR-6	Scalability	The storage layer is accessed exclusively through a StorageService interface, allowing LocalStorage to be replaced by IndexedDB or a hosted database without modification to any component or analytics engine. Aggregations are single-pass and scale linearly with transaction count.
NFR-7	Error Handling	Client-side validation blocks non-positive amounts, invalid dates, and empty required fields. CSV import validates every row and reports rejected rows with line numbers while still importing valid records. All storage operations are wrapped in guarded blocks with non-blocking user notifications.
NFR-8	Resource Efficiency	No external charting library is bundled — donut and bar charts are hand-authored SVG, substantially reducing bundle size. Analytics results are memoised and recomputed only when the transaction set changes.

5. System Architecture

SpendSense follows a three-layer client-side architecture. The presentation layer contains React components responsible only for rendering and user interaction. The business logic layer contains independent, framework-agnostic service engines — the natural-language parser, personality engine, anomaly detector, budget calculator, and statistics service — each of which is a pure module operating on plain data. The data layer wraps browser persistence and CSV interchange behind a single storage service interface.

The key architectural property is that no component talks to storage directly and no engine depends on React. Components request computed results from engines; engines read data through the storage service. This keeps every analytical rule unit-testable in isolation and allows the persistence mechanism to be replaced without touching the interface.

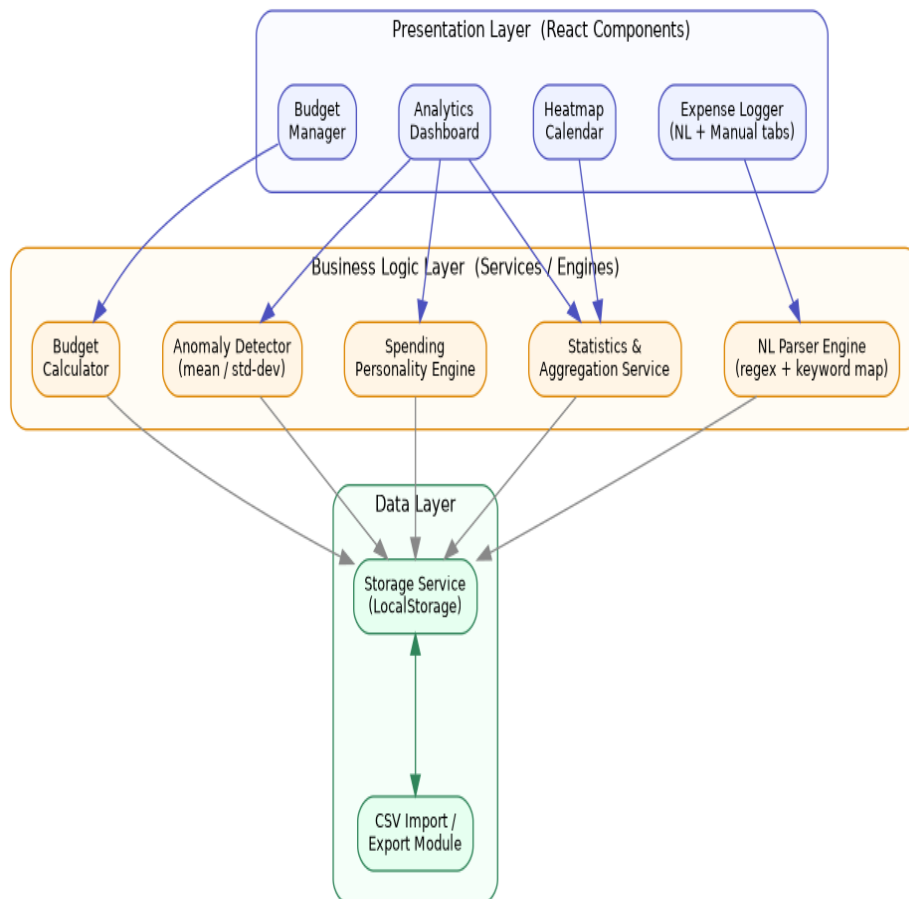


Figure 1: Three-layer System Architecture

6. Design Diagrams

6.1 Use Case Diagram

The Student is the sole actor; the application has no external system dependency, since all parsing and analysis is performed locally. The diagram enumerates the complete set of user-initiated interactions across the three functional modules.

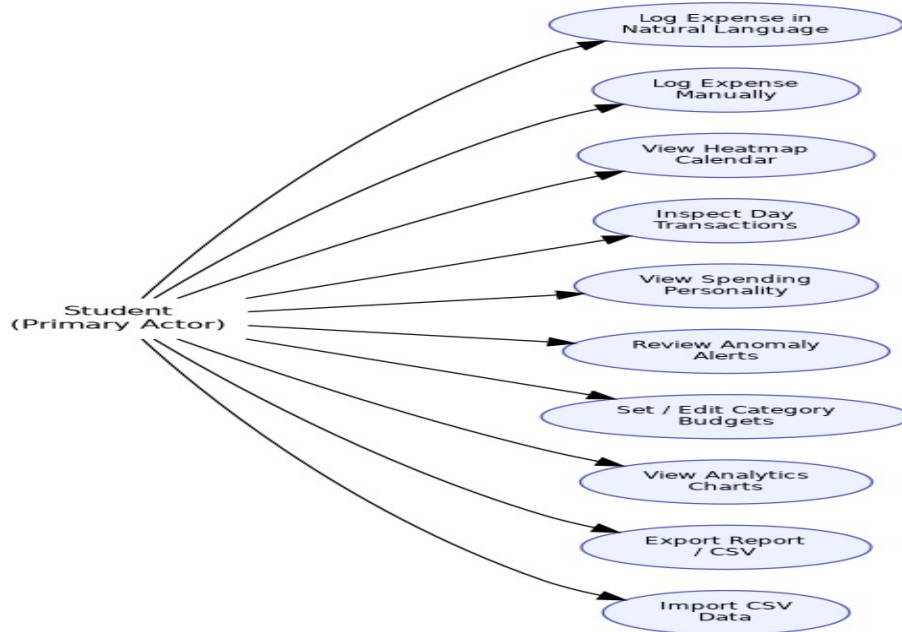


Figure 2: Use Case Diagram

6.2 Workflow / Process Flow Diagram

The workflow shows the dual-mode entry path and the validation loop. Whichever path is used, a validated record is persisted, all analytics engines are re-run, and the dashboard and alerts are updated — after which the student may optionally export a report or CSV.

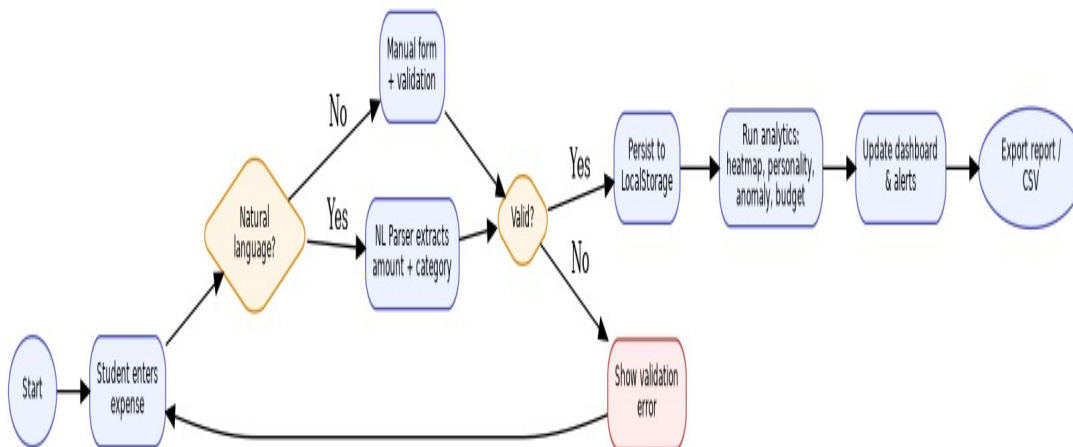


Figure 3: Process Workflow Diagram

6.3 Sequence Diagram

The sequence below traces a single natural-language logging interaction end to end, from the typed sentence through parsing, validation, and persistence, to the recomputation of insights and the re-rendering of the dashboard.

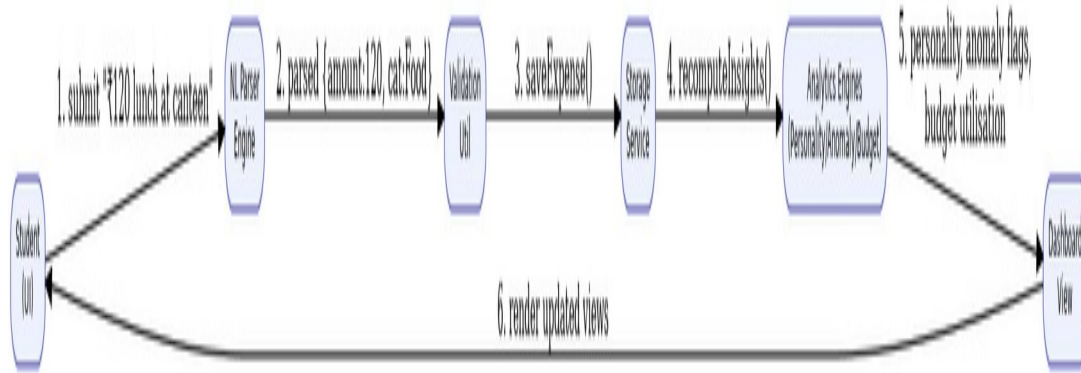


Figure 4: Sequence Diagram — Natural Language Expense Logging

6.4 Class / Component Diagram

The domain model centres on the Expense entity, which every engine consumes. Category is an enumeration of the eight student-tailored types. Each analytical engine — NLPParserService, PersonalityEngine, AnomalyDetector, StatisticsService — is an independent service class, and StorageService is the single persistence boundary.

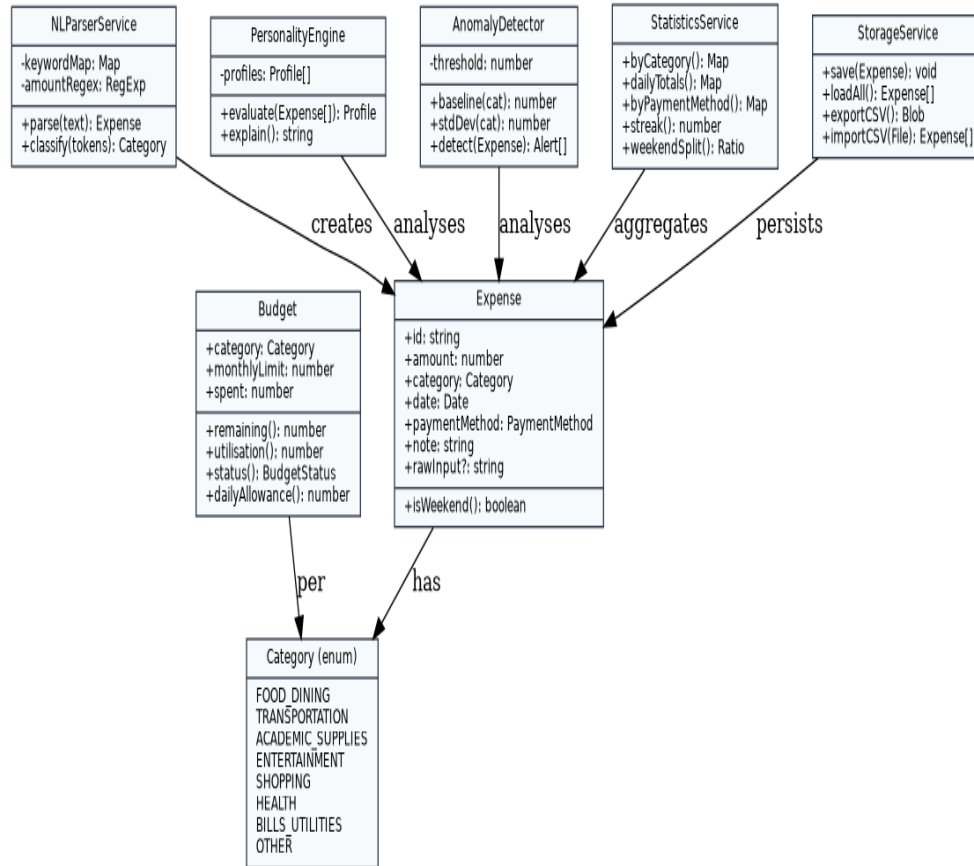


Figure 5: Class / Component Diagram

6.5 ER Diagram (Storage Design)

Although the current implementation persists to LocalStorage as serialised JSON, the data is modelled relationally so that migration to IndexedDB or a hosted relational database is straightforward. The schema below reflects that logical structure: expenses reference a category and a payment method, budgets are defined per category per month, and anomaly alerts reference the expense that triggered them.

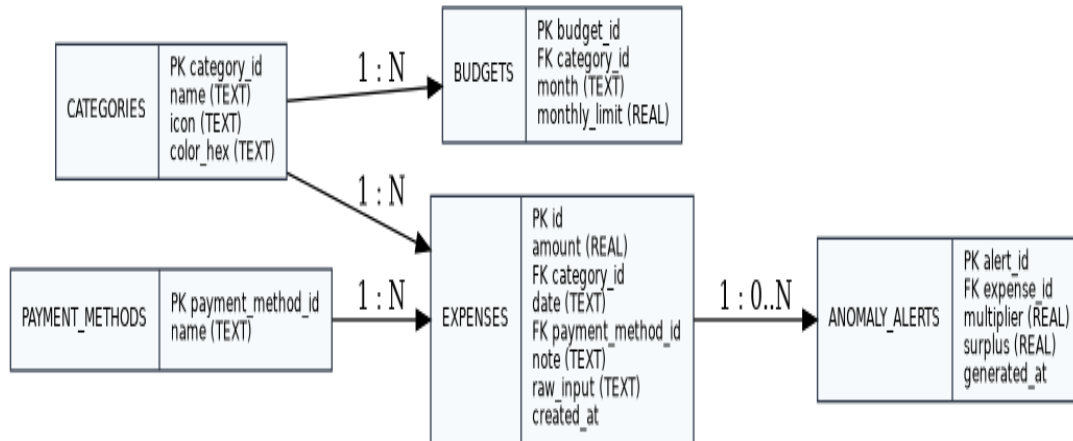


Figure 6: Entity-Relationship Diagram

Schema summary:

```

EXPENSES(id PK, amount, category_id FK, date, payment_method_id FK, note,
raw_input, created_at)
CATEGORIES(category_id PK, name, icon, color_hex)
PAYMENT_METHODS(payment_method_id PK, name)
BUDGETS(budget_id PK, category_id FK, month, monthly_limit)
ANOMALY_ALERTS(alert_id PK, expense_id FK, multiplier, surplus, generated_at)
  
```

7. Design Decisions & Rationale

7.1 Rule-based parsing rather than a machine-learning model

Category inference could have been implemented with a trained classifier. A regex-and-keyword-map approach was chosen deliberately for three reasons. First, no labelled dataset of Indian student expense phrasings exists, and collecting one was outside the project's scope. Second, the vocabulary is small and highly predictable — canteen, mess, xerox, bus, auto, hostel, wifi — so a curated keyword map achieves high accuracy on the target domain without training. Third, and most importantly, a rule-based engine is fully explainable: when the student sees a category they disagree with, the reason is inspectable and correctable, whereas a model's misclassification is opaque. Explainability was treated as a product requirement, not an implementation detail.

7.2 Rule-based personality assignment with visible rationale

The Spending Personality Engine was intentionally built on transparent ratio thresholds rather than clustering. The feature's value lies in the student believing and acting on the result, which requires showing the arithmetic — "42% of your spending is Food & Dining" — alongside the label. A black-box assignment would carry the same label with none of the persuasive force, and would be impossible to defend if questioned during evaluation.

7.3 Standard deviation over a fixed threshold for anomaly detection

A flat rupee threshold would be meaningless across categories of very different scale: ₹2,000 is unremarkable for Bills & Utilities and extraordinary for Academic Supplies. Computing a per-category mean and standard deviation makes the detector self-calibrating to each student's actual habits, so a heavy commuter is not warned about every bus pass while a light spender is still alerted to a genuine outlier. Reporting the multiplier and surplus rather than a raw flag keeps the alert informative and non-judgemental.

7.4 Eight custom categories instead of generic financial categories

The category set was derived directly from the target user's actual spending surface — campus food, commuting, academic supplies, hostel bills — rather than adopted from mainstream finance applications. This is the single decision that most affects analytical usefulness: generic schemes route the majority of student transactions into "Other", which makes both the personality engine and the budget module uninformative.

7.5 LocalStorage and a fully client-side design

Financial data is sensitive and students are reasonably reluctant to upload it. Keeping everything on-device removed the need for authentication, a backend, and a privacy policy, eliminated an entire class of security risk, and allowed the application to work offline. The cost — no cross-device synchronisation — was accepted as the correct trade-off for this user group, and the StorageService abstraction preserves the option to add a backend later.

7.6 Hand-authored SVG charts instead of a charting library

Charting libraries impose significant bundle weight and constrain styling. Because only two chart types were required, both were implemented directly in SVG. This kept the bundle small, gave exact control over slice highlighting and the heatmap intensity scale, and demonstrated the underlying geometry and data-to-coordinate mapping rather than delegating it.

7.7 Dual-mode entry

Natural-language entry alone would be fragile — unusual phrasings and ambiguous amounts will occasionally fail. Rather than trying to make the parser exhaustive, the design guarantees a fallback: any unparseable input is redirected to the structured form with the original text preserved. This converts parser failure from a data-loss event into a minor inconvenience.

8. Implementation Details

The application is implemented in React with TypeScript, built with Vite and styled with Tailwind CSS. The codebase is organised into components, services, models, and utilities, comfortably exceeding the brief's requirement of five to ten meaningful modules.

8.1 Module Inventory

Module / File	Responsibility
components/ExpenseLogger.tsx	Dual-tab entry interface — natural language input and structured manual form with validation feedback.
components/HeatmapCalendar.tsx	Monthly calendar grid, five-tier intensity colouring, and interactive day inspector.
components/PersonalityCard.tsx	Renders the assigned spending profile with ratio breakdown and advice.
components/AnomalyAlerts.tsx	Displays flagged transactions with multiplier and surplus figures.
components/BudgetManager.tsx	Category limit editing, utilisation bars, status badges, daily allowance display.
components/charts/DonutChart.tsx	Hand-authored SVG donut chart with arc computation and slice highlighting.
components/charts/VelocityBarChart.tsx	Daily spending velocity bar chart with axis scaling.
services/nlParser.ts	Regex amount extraction, tokenisation, and keyword-map category inference.
services/personalityEngine.ts	Ratio computation and threshold-based profile assignment with rationale generation.
services/anomalyDetector.ts	Per-category mean and standard deviation baselines; outlier detection and alert construction.
services/budgetService.ts	Remaining balance, utilisation percentage, status classification, daily allowance calculation.
services/statisticsService.ts	Category, daily, and payment-method aggregations; streaks; weekend/weekday split.
services/storageService.ts	LocalStorage persistence plus CSV export and validated CSV import.
models/types.ts	Expense, Category, PaymentMethod, Budget, Alert, and Profile type definitions.
utils/validation.ts	Amount, date, and required-field validation rules.
utils/dateUtils.ts	Month boundaries, days remaining, weekend detection, streak calculation.

8.2 Natural Language Parsing

Parsing proceeds in three stages. The amount is extracted first, using a regular expression that tolerates an optional currency symbol and decimal component, so "₹120", "120", and "120.50" are all recognised. The remaining text is then lowercased and tokenised on whitespace and punctuation. Each token is looked up in a keyword map that associates domain vocabulary with categories — canteen, mess, lunch, chai map to Food & Dining; bus, auto, metro, ticket to Transportation; xerox, textbook, stationery, printout to Academic Supplies; wifi, hostel, electricity, recharge to Bills & Utilities. The category with the highest token match count wins; where no token matches, the expense is assigned to Other and the entry is surfaced for the student to confirm.

```

parse("₹120 lunch at college canteen")
→ { amount: 120, category: FOOD_DINING, rawInput: "₹120 lunch at college canteen" }

```

8.3 Anomaly Detection Logic

For each category, the detector computes the arithmetic mean and standard deviation of historical amounts. A new expense is flagged when it exceeds the category mean by more than the configured multiple of the standard deviation. The alert reports the multiplier relative to the baseline mean and the absolute surplus above it — for instance, a ₹2,450 purchase against a ₹180 Academic Supplies baseline is reported as roughly 13.6× the baseline with a surplus of ₹2,270. A minimum sample size is enforced before a category is eligible for detection, preventing spurious alerts from a near-empty history.

8.4 Heatmap Intensity Mapping

Daily totals are mapped onto five discrete tiers rather than a continuous gradient, because discrete bands are easier to read at a glance and easier to compare across months. The thresholds — None, Low up to ₹150,

Moderate up to ₹400, High up to ₹850, and Surge above ₹850 — were chosen to reflect realistic student daily spending, and are declared as configuration constants so they can be tuned without touching rendering code.

8.5 Budget Calculations

For each category the service computes remaining balance as limit minus spend, and utilisation as spend divided by limit. Status is classified into four bands — Safe, Moderate, Near Limit, and Exceeded — driving the colour-coded badge. The recommended daily allowance divides the total remaining balance by the number of days left in the current month, giving the student a concrete, forward-looking figure rather than a retrospective total.

8.6 Validation and Error Handling

Validation is centralised in a single utility module used by both entry paths, guaranteeing identical rules regardless of how an expense is entered. Amounts must be positive and numeric; dates must be valid and not in the future; category and payment method must be members of their respective enumerations. On CSV import, each row is validated independently — malformed rows are collected and reported with their line numbers while all valid rows are still imported, so a single bad line never aborts an entire import.

9. Screenshots / Results

Insert screenshots of the running application in this section. The following captures are recommended, as they collectively evidence all three functional modules and the supporting analytics:

- Natural language logger — showing an entry such as "₹120 lunch at college canteen" being parsed and the inferred category displayed.
- Manual logging tab — showing the structured form and a triggered validation error for an invalid amount.
- Heatmap calendar — a populated month showing the full range of intensity tiers including at least one Surge day.
- Day inspector — the expanded transaction breakdown for a selected calendar cell.
- Spending Personality card — the assigned profile with its ratio rationale and advice.
- Anomaly alert panel — a flagged transaction showing the multiplier and surplus.
- Budget manager — category limits with utilisation bars across all four badge states and the recommended daily allowance.
- Analytics dashboard — the SVG donut chart, daily velocity bar chart, and payment-method breakdown.
- Generated report / printable statement and a successful CSV export or import confirmation.

10. Testing Approach

Testing combined automated unit tests over the pure logic engines with structured manual validation of the interface. Because all analytical engines are framework-agnostic modules operating on plain data, they are testable without rendering any component.

10.1 Unit Tests

- Parser — correct amount extraction with and without a currency symbol, with decimals, and when the amount appears mid-sentence.
- Classifier — each of the eight categories triggered by representative vocabulary; unmatched input correctly falls through to Other.
- Anomaly detector — baseline mean and standard deviation computed correctly; a known outlier is flagged with the expected multiplier and surplus; no alerts are raised below the minimum sample size.
- Budget service — remaining balance, utilisation percentage, correct badge state at each threshold boundary, and daily allowance across month-end edge cases.
- Statistics service — category and daily totals, weekend/weekday split, and streak computation including breaks in the streak.
- Validation utilities — rejection of zero, negative, and non-numeric amounts, and of invalid or future dates.

10.2 Manual Validation

#	Test	Expected Result
1	Enter "₹120 lunch at college canteen"	Amount 120, category Food & Dining
2	Enter "250 xerox and textbooks"	Amount 250, category Academic Supplies
3	Enter "1800 hostel wifi bills"	Amount 1800, category Bills & Utilities
4	Enter text with no recognisable amount	Redirected to manual form with text preserved
5	Submit manual form with a negative amount	Validation error shown; record not saved
6	Log expenses totalling over ₹850 on one date	Calendar cell renders in the Surge tier
7	Click a populated calendar cell	Day inspector lists that day's transactions
8	Log ₹2,450 in a low-baseline category	Anomaly alert with correct multiplier and surplus
9	Log food-dominant history, open personality panel	Resolves to The Campus Foodie with visible rationale
10	Set a category limit below current spend	Badge switches to Exceeded
11	Export CSV, clear data, re-import the file	All records restored with categories intact
12	Import a CSV containing one malformed row	Valid rows imported; malformed row reported by line

#	Test	Expected Result
		number
13	Refresh the browser	All previously logged data persists

11. Challenges Faced

11.1 Ambiguity in natural language amount extraction

Early parser versions misread entries containing more than one number — for example "2 coffees 80" or a date embedded in the text. The initial regex captured the first number encountered, which was frequently a quantity rather than a price. This was resolved by prioritising tokens carrying a currency symbol, and otherwise selecting the largest plausible numeric token, which correctly handles the common quantity-then-price phrasing pattern.

11.2 Keyword collisions across categories

Several vocabulary items legitimately belong to more than one category. The word "book" may indicate Academic Supplies or a booking under Transportation; "paper" may be stationery or an examination fee. A single-match lookup produced unpredictable results. The classifier was changed to score all matching tokens per category and select the highest-scoring category, with a deterministic tie-break order, which made behaviour both more accurate and reproducible.

11.3 Anomaly detection on sparse history

With only two or three transactions in a category, the standard deviation is extremely small and almost any new expense appears anomalous — the detector generated a flood of false alerts for new users. A minimum sample-size requirement was introduced before a category becomes eligible for detection, and the detector now stays silent rather than producing alerts it cannot statistically justify.

11.4 Personality thresholds producing unstable profiles

The first threshold set caused the assigned profile to flip between labels after a single new expense, which undermined confidence in the feature. Thresholds were widened and a dominance margin introduced, so a category must lead by a meaningful gap before it determines the profile; where no category leads clearly, The Balanced Manager is assigned rather than forcing a specialised label.

11.5 Rendering the donut chart without a charting library

Constructing the SVG arc path data manually required deriving the correct polar-to-Cartesian conversion and handling the large-arc-flag for slices exceeding half the circle, which initially produced inverted or invisible segments. Working through the arc geometry directly proved to be one of the more instructive parts of the project.

11.6 Calendar boundary and timezone handling

Aligning the first day of a month to the correct weekday column, and correctly grouping transactions by local date, produced off-by-one errors near month boundaries when date handling was inconsistent. Centralising all date logic in a single utility module with a consistent local-date representation resolved the issue.

12. Learnings & Key Takeaways

- Reducing input friction is a design problem, not a UI problem. The project's central insight is that an expense tracker's analytical quality is capped by the completeness of its data, and completeness is determined by how little effort each entry costs. Every downstream feature depends on that one decision.
- Explainability is a feature, not an implementation detail. Choosing rule-based engines over opaque models made every output defensible and correctable, and made the personality and anomaly features persuasive in a way a black-box score could not have been.
- Statistical baselines must be self-calibrating. A fixed threshold would have been meaningless across categories of different scale; using per-category mean and standard deviation taught the practical value of normalising against context rather than against an absolute.
- Separating pure logic from rendering pays for itself in testing. Because the engines carry no framework dependency, the entire analytical surface of the application is unit-testable without rendering a single component.

- Edge cases in statistics appear at low sample sizes. The sparse-history false-alert problem was a concrete lesson in the difference between an algorithm that is mathematically correct and one that behaves sensibly on real, incomplete data.
- Implementing visualisation primitives directly builds genuine understanding. Deriving SVG arc geometry by hand taught more about how charts actually work than configuring a library would have.

13. Future Enhancements

- Replace LocalStorage with IndexedDB or a hosted backend behind the existing StorageService interface, enabling cross-device synchronisation without changes to components or engines.
- Add a correction-learning loop so that when a student re-categorises a parsed expense, the keyword map is updated, allowing the classifier to adapt to individual vocabulary over time.
- Support recurring expense templates for predictable monthly outflows such as hostel fees and mobile recharges.
- Add SMS or UPI notification parsing on mobile to capture transactions automatically, reducing entry effort further.
- Introduce multi-month trend analysis with month-over-month category comparison and simple forecasting of month-end totals.
- Add configurable savings goals with progress tracking, alongside the existing expenditure-focused budgets.
- Extend the personality engine to track profile evolution over time, showing the student how their spending identity shifts across semesters.
- Improve accessibility with full keyboard navigation of the calendar grid and ARIA labelling of chart elements.

14. References

1. React Documentation — Official Guide. <https://react.dev>
2. TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html>
3. Vite Documentation — Build Tooling Guide. <https://vitejs.dev/guide/>
4. Tailwind CSS Documentation. <https://tailwindcss.com/docs>
5. MDN Web Docs — SVG Paths and Arc Commands. <https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths>
6. MDN Web Docs — Regular Expressions Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions
7. MDN Web Docs — Window.localStorage API. <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
8. RFC 4180 — Common Format and MIME Type for Comma-Separated Values (CSV) Files. <https://www.rfc-editor.org/rfc/rfc4180>
9. Course lecture notes and syllabus material — Programming in Java.
10. VITYarhi — Build Your Own Project: General Project Instructions & Submission Guidelines.